

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Traversal Challenge

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO3 – Precision (BL3, BL5)	Assessment:	Assessment Tool 2 – Traversal Challenge
Weightage:	30% of CIA	Total Marks:	100
CO3 Statement:	Solve problems for optimized data storage and retrieval using Binary Search Trees, AVL Trees, B-Trees, Red-Black Trees, and Splay Trees.		
Student Name:	Sree B	Reg. No.:	192524023
Section / Batch:	AI and DS / 2025	Date:	03/08/2026

Code of Conduct

I Sree B (192524023) _____ (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sree B _____

Instructions

- This assessment tool contains 5 Traversal Challenge questions, Total: 100 Marks.
- Students can use any online/offline Traversal Challenge. Demonstrate tree.
- When a student answer using simulator, in evaluation the answer should have captured screenshots after every major operation
- Submit the report in PDF format with properly labelled screenshots as proof of completion.

Section / Topic		Focus	Q Nos	Marks
Tree Traversals		Traversal types, In order, Post order and Pre order	5	100
GRAND TOTAL:			100 Marks	

Traversal Challenge

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	20	Demonstrates thorough understanding of all core Data Structures concepts; answers accurately reflect deep knowledge	Shows good understanding with minor conceptual gaps	Basic understanding; several misconceptions present	Limited understanding; major concepts misunderstood
Traversal Accuracy	20	Correctly completes all tree traversals (Preorder, Inorder, Postorder, Level Order) without errors.	Completes most traversals correctly with one minor error.	Correctly performs only some traversals with multiple errors.	Unable to perform most traversals correctly.
Selection of Appropriate Traversal	30	Correctly selects the appropriate traversal for every given problem scenario.	Selects the correct traversal for most scenarios.	Selects appropriate traversal for only a few scenarios.	Frequently selects incorrect traversal methods.
Completion & Time Efficiency	20	Completes all challenges within the allotted time while maintaining accuracy.	Completes most challenges within the time limit.	Completes only part of the challenge or exceeds the time limit slightly.	Unable to complete the challenge within the allotted time.
Evidence Submission	10	Uploads complete screenshots showing successful completion, score, and student details.	Uploads screenshots with minor missing information.	Uploads incomplete or unclear screenshots.	No valid evidence submitted.

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Write the traversal sequences for following trees

Data Structure Visualizer

tree-graph-algorithm-visualizer.vercel.app/treetraversals

Tree Traversals

Controls

Input
Input is level-by-level order.

100,20,200,10,30,150,300

Enter numbers separated by commas. Use 'null' for empty nodes.

Traversal Order

Pre In Post

Generate Default

Visualize

Start Simulation

10 30 20 150 300 200 100

```
graph TD; 100((100)) --> 20((20)); 100 --> 200((200)); 20 --> 10((10)); 20 --> 30((30)); 200 --> 150((150)); 200 --> 300((300));
```

Data Structure Visualizer

tree-graph-algorithm-visualizer.vercel.app/treetraversals

Tree Traversals

Controls

Input
Input is level-by-level order.

100,20,200,10,30,150,300

Enter numbers separated by commas. Use 'null' for empty nodes.

Traversal Order

Pre In Post

Generate Default

Visualize

Start Simulation

10 20 30 100 150 200 300

```
graph TD; 100((100)) --> 20((20)); 100 --> 200((200)); 20 --> 10((10)); 20 --> 30((30)); 200 --> 150((150)); 200 --> 300((300));
```

tree-graph-algorithm-visualizer.vercel.app/treetraversals

Ask Gemini | School

Back to Home

Tree Traversals

?

Controls

Input

Input is level-by-level order.

100,20,200,10,30,150,300

Enter numbers separated by commas. Use 'null' for empty nodes.

Traversal Order

PreInPost

Generate Default

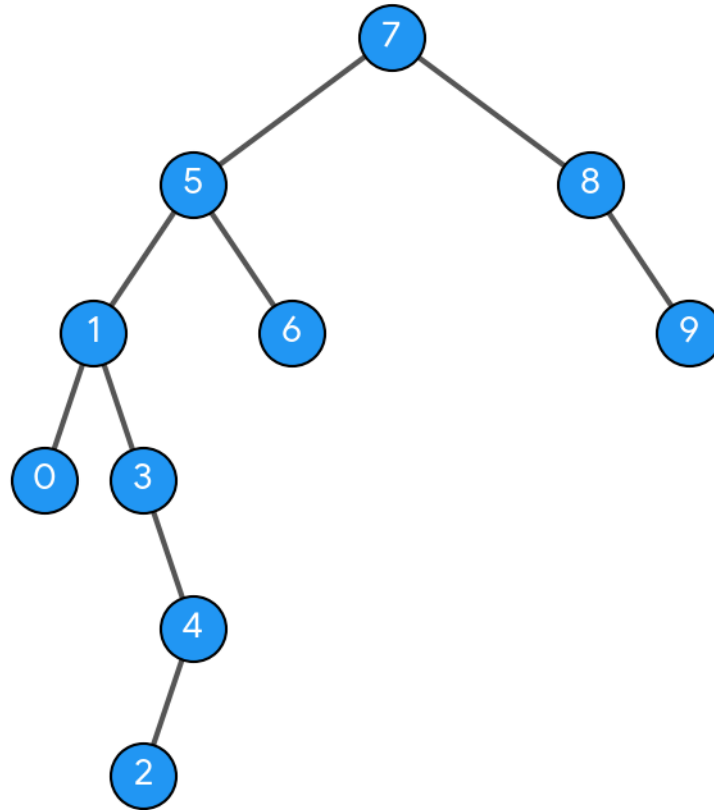
Visualize

Start Simulation

100201030200150300

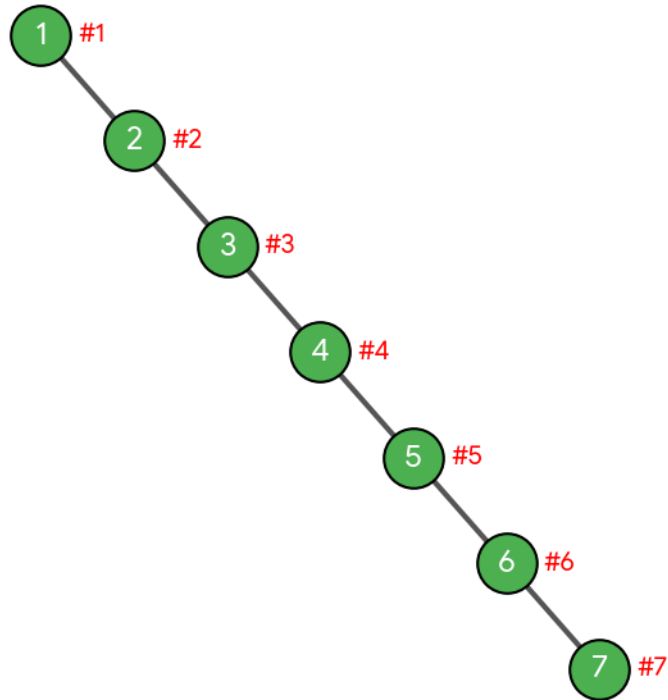
```
graph TD; 100((100)) --- 20((20)); 100 --- 200((200)); 20 --- 10((10)); 20 --- 30((30)); 200 --- 150((150)); 200 --- 300((300)); style 300 fill:#f96
```

2. Assume the digits 7, 5, 1, 8, 3, 6, 0, 9, 4, 2 are added in that sequence into a binary search tree that is initially empty. The binary search tree follows the standard natural number ordering. What is the resulting tree's traversal sequence?

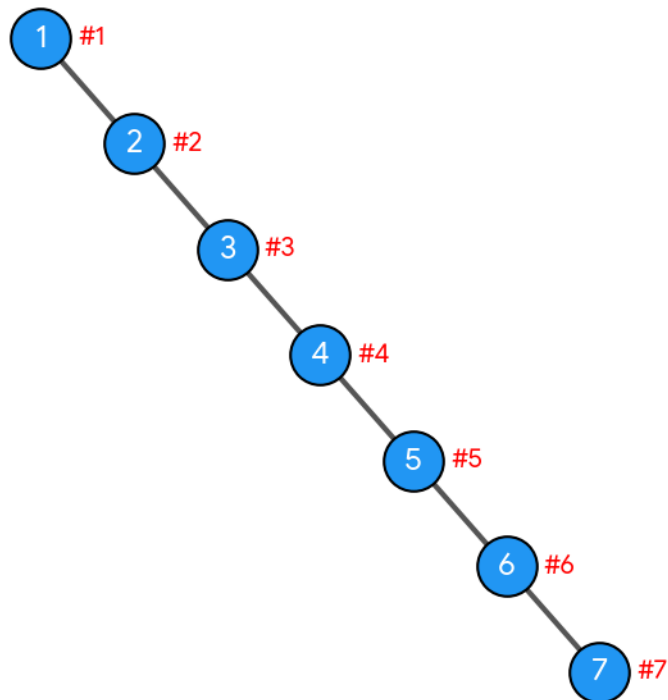


3. Write the Inorder, Preorder & Postorder traversal sequences for following trees

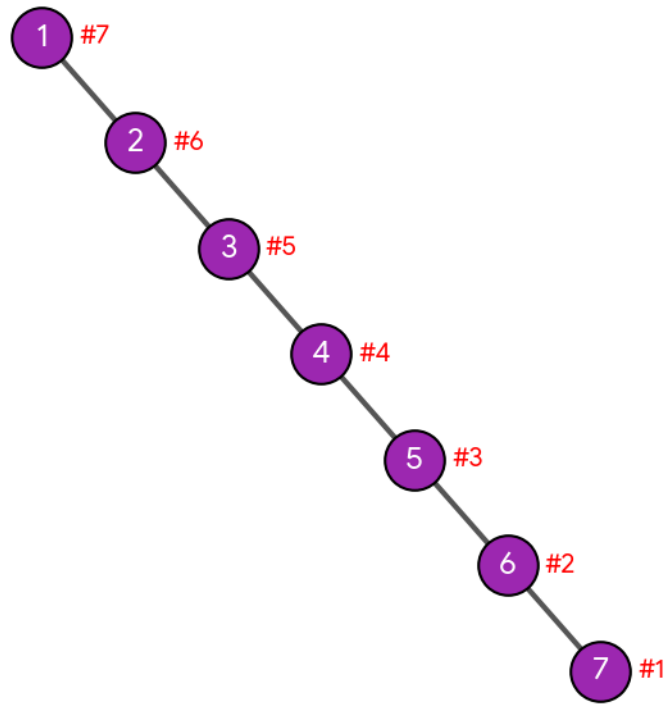
Pre-Order Traversal Order (#)



In-Order Traversal Order (#)

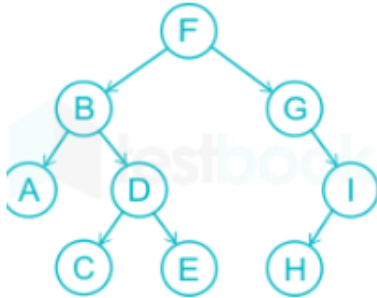


Post-Order Traversal Order (#)



Question No: 4

What is the sequence of nodes when applying in-order traversal on the binary tree given below?



1. A, B, C, D, E, F, I, H, G

2. A, C, D, E, B, F, G, H, I

3. A, B, C, D, E, F, G, H, I

4. I, H, G, F, E, D, C, B, A

The correct option is **3. A, B, C, D, E, F, G, H, I.**

Step-by-Step Breakdown

In-order traversal follows the rule: **Left Subtree** $\rightarrow$ **Root** $\rightarrow$ **Right Subtree**.

1. **Left Subtree of F (rooted at B):**
 - Go to the leftmost node: **A**
 - Visit root: **B**
 - Traverse right subtree of B (rooted at D):
 - Left child: **C**
 - Root: **D**
 - Right child: **E**
 - *Sequence so far:* A, B, C, D, E
2. **Visit the Main Root: F**
 - *Sequence so far:* A, B, C, D, E, F
3. **Right Subtree of F (rooted at G):**
 - Left child of G: None
 - Visit root: **G**
 - Traverse right subtree of G (rooted at I):
 - Left child: **H**
 - Root: **I**
 - *Sequence so far:* G, H, I

Final Sequence

Combining all steps gives: **A, B, C, D, E, F, G, H, I**

answer the question

The correct option is **3. A, B, C, D, E, F, G, H, I.**

Step-by-Step Breakdown

In-order traversal follows the rule: **Left Subtree $\rightarrow$ Root $\rightarrow$ Right Subtree.**

1. **Left Subtree of F (rooted at B):**
 - Go to the leftmost node: **A**
 - Visit root: **B**
 - Traverse right subtree of B (rooted at D):
 - Left child: **C**
 - Root: **D**
 - Right child: **E**
 - *Sequence so far:* A, B, C, D, E
2. **Visit the Main Root: F**
 - *Sequence so far:* A, B, C, D, E, F
3. **Right Subtree of F (rooted at G):**
 - Left child of G: None
 - Visit root: **G**
 - Traverse right subtree of G (rooted at I):
 - Left child: **H**
 - Root: **I**
 - *Sequence so far:* G, H, I

Final Sequence

Combining all steps gives: **A, B, C, D, E, F, G, H, I**

If you'd like, I can also show you how the **pre-order** or **post-order** traversals would look for this same tree. Assume the digits 7, 5, 1, 8, 3, 6, 0, 9, 4, 2 are added in that sequence into a binary search tree that is initially empty. The binary search tree follows the standard natural number ordering. What is the resulting tree's traversal sequence generate the image for this question

To find the traversals, we first construct the Binary Search Tree (BST) by inserting the numbers 7, 5, 1, 8, 3, 6, 0, 9, 4, 2 one by one using standard ordering (smaller values to the left, larger values to the right):

- 7: Root
- 5: Left of 7
- 1: Left of 5
- 8: Right of 7
- 3: Right of 1
- 6: Right of 5
- 0: Left of 1
- 9: Right of 8
- 4: Right of 3
- 2: Left of 4

Here is the visual structure of the resulting BST:

Traversal Sequences

Depending on the traversal method required, the sequences are:

- **In-Order** (Left $\rightarrow$ Root $\rightarrow$ Right): **0, 1, 2, 3, 4, 5, 6, 7, 8, 9** (*Always sorted for any valid BST*)
- **Pre-Order** (Root $\rightarrow$ Left $\rightarrow$ Right): **7, 5, 1, 0, 3, 4, 2, 6, 8, 9**
- **Post-Order** (Left $\rightarrow$ Right $\rightarrow$ Root): **0, 2, 4, 3, 1, 6, 5, 9, 8, 7**

If you want, let me know if you would like me to list out the **step-by-step trace** for any specific one of these traversals!

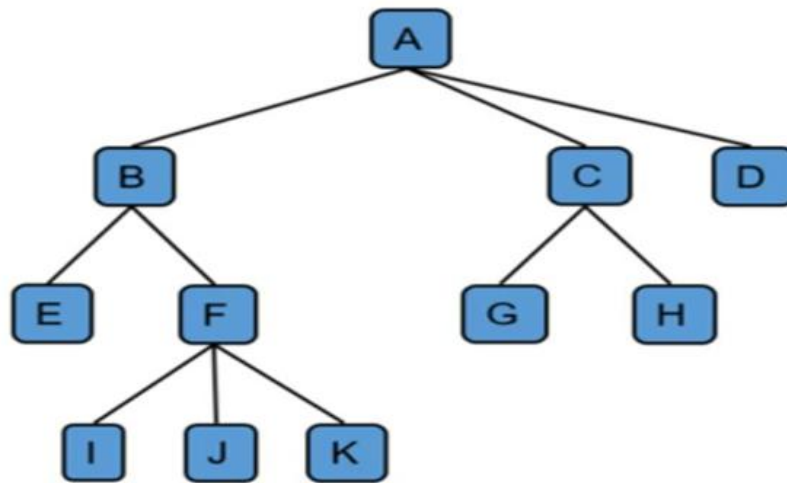
write inorder, preorder, postorder for 1,2,3,4,5,6,7 generate the images separately

Inserting the numbers 1, 2, 3, 4, 5, 6, 7 into an initially empty Binary Search Tree sequentially creates a **right-skewed (degenerate) tree**, where every node only has a right child.

1. Pre-Order Traversal (Root $\rightarrow$ Left $\rightarrow$ Right)

Sequence: 1, 2, 3, 4, 5, 6, 7

Question No: 5



1. **1.Traverse Leftmost Subtree of root A (rooted at B):**

- Leftmost child of B is E.
- Visit the parent root: **B**.
- Traverse remaining child subtrees of B (rooted at F):
 - Process leftmost child of F: **I**.
 - Visit the parent node: **F**.
 - Process next children from left to right: **J**, then **K**.
- *Resulting segment:* E, B, I, F, J, K

2. **2.Visit Main Root Node: A**

- *Combined segment:* E, B, I, F, J, K, A

3. **3.Traverse Remaining Subtrees of A (rooted at C and D):**

- **Subtree under C:**
 - Process leftmost child of C: **G**.
 - Visit parent node: **C**.
 - Process right child of C: **H**.
- **Node D:** Has no children, simply visit **D**.
- *Resulting segment:* G, C, H, D

Final Combined Sequence

E, B, I, F, J, K, A, G, C, H, D

1. Pre-Order Traversal (Root $\rightarrow$ Left to Right Subtrees)

Rule: Visit the parent node first, then move through its child subtrees from left to right.

1. **Visit main root: A**

2. **Move to A's first child (B) and its subtree:**

- Visit B
- Visit B's first child: E
- Visit B's second child (F) and its subtree:
 - Visit F
 - Visit F's children from left to right: I, J, K
- *Subtree B path completed:* B, E, F, I, J, K

3. **Move to A's second child (C) and its subtree:**

- Visit C
- Visit C's children from left to right: G, H
- *Subtree C path completed:* C, G, H

4. **Move to A's third child (D):**

- Visit D

Complete Pre-Order Sequence:

A, B, E, F, I, J, K, C, G, H, D

2. Post-Order Traversal (Left to Right Subtrees $\rightarrow$ Root)

Rule: Completely traverse all child subtrees from left to right before visiting their parent node.

1. **Explore A's first child (B) and its subtree completely first:**

- Go down to B's first child: E (has no children, so record E)
- Go to B's second child (F) and clear its children first: I, J, K
- Record parent of those children: F
- Now that all of B's children are cleared, record parent: B
- *Subtree B path completed:* E, I, J, K, F, B

2. **Explore A's second child (C) and its subtree completely:**

- Clear C's children from left to right: G, H
- Record parent: C
- *Subtree C path completed:* G, H, C

3. **Explore A's third child (D):**

- Has no children, so record D

4. **Finally, record the main root node:**

- Visit A

Complete Post-Order Sequence:

E, I, J, K, F, B, G, H, C, D, A